ASSIGNMENT 11

Terraform Installation in Local

Terraform is an open-source Infrastructure as Code (IaC) tool from HashiCorp, allowing users to define, provision, and manage cloud (AWS, Azure, GCP) and on-premises infrastructure using a declarative language (HCL) in configuration files, enabling automation, versioning, and efficient, consistent management of resources like VMs, networks, and storage across different environments.

Installation of Terraform:

1. **Terraform as a Binary**

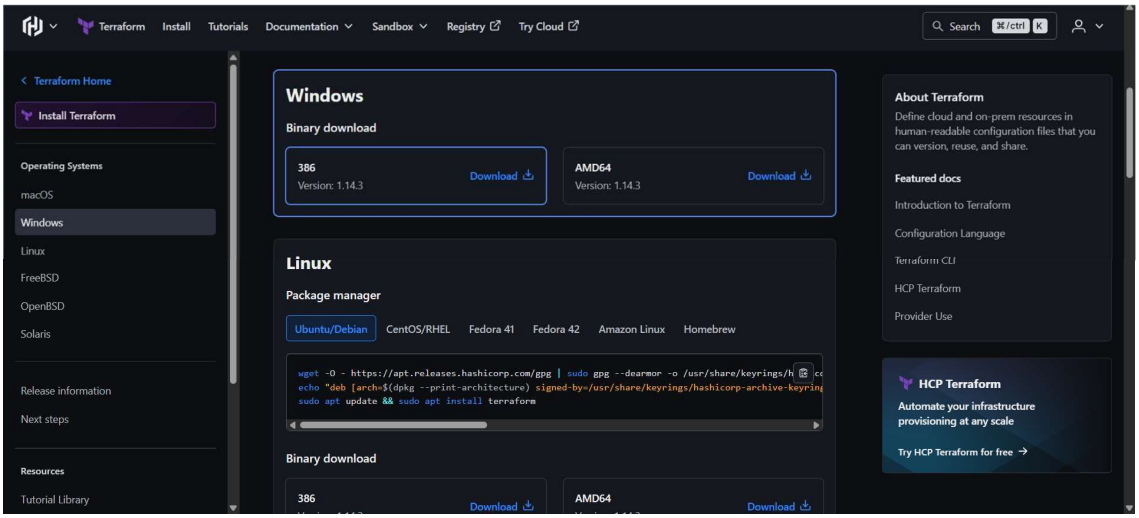
Terraform is distributed as a single compiled binary:

- terraform.exe on Windows
- terraform on Linux/macOS

There is no installer wizard, no background process, and no daemon.
When you install Terraform, you are:

- Downloading the binary
- Making it accessible system-wide

That's it.

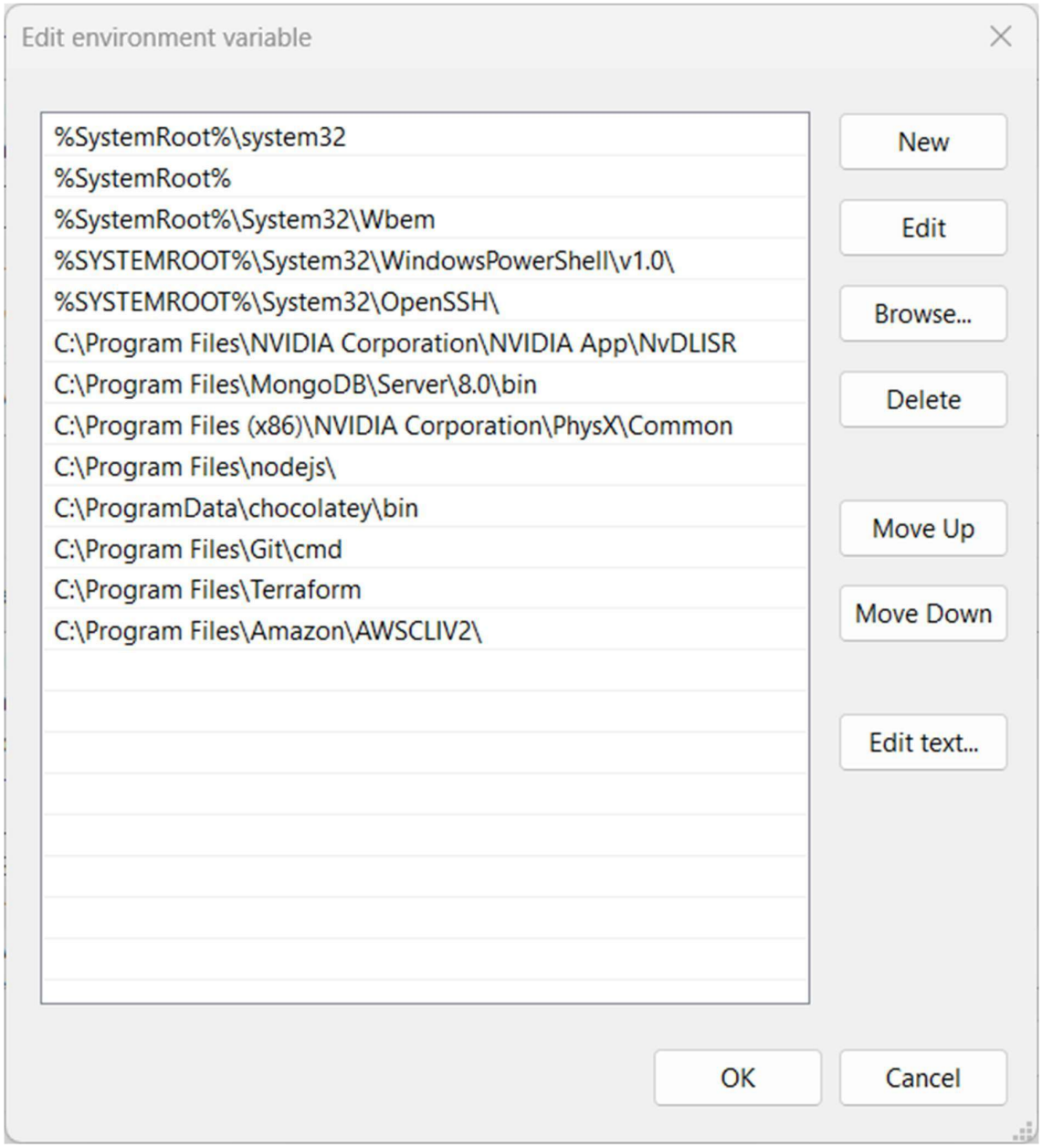


2. Role of the Operating System PATH

Operating systems locate executable programs using the PATH environment variable.

- PATH is a list of directories
- When you type terraform, the OS searches those directories in order
- If Terraform's directory is in PATH → command runs
- If not → "command not found"

So conceptually, installation = placing the Terraform binary in a directory listed in PATH, in (**system environment).**



3. Package Managers (Why They Exist)

Package managers (Chocolatey, APT, Homebrew) automate three things:

1. Downloading the correct Terraform binary for your OS
2. Placing it in a standard system directory
3. Updating PATH automatically

They do nothing magical beyond that.
They just remove human error.

4. Verification Concept

After installation, running Terraform's version check serves one purpose:

- Confirms the OS can locate and execute the binary
- Confirms the binary is intact and runnable

If verification fails, the installation conceptually failed — not Terraform.

```
C:\Windows\System32>terraform --version
Terraform v1.14.3
on windows_386
```

AWS Command Line Interface:

1. What AWS CLI Actually Is AWS CLI is:

- A command-line client
- A wrapper around AWS REST APIs
- A program that converts human-readable commands into HTTPS API calls

Conceptually:

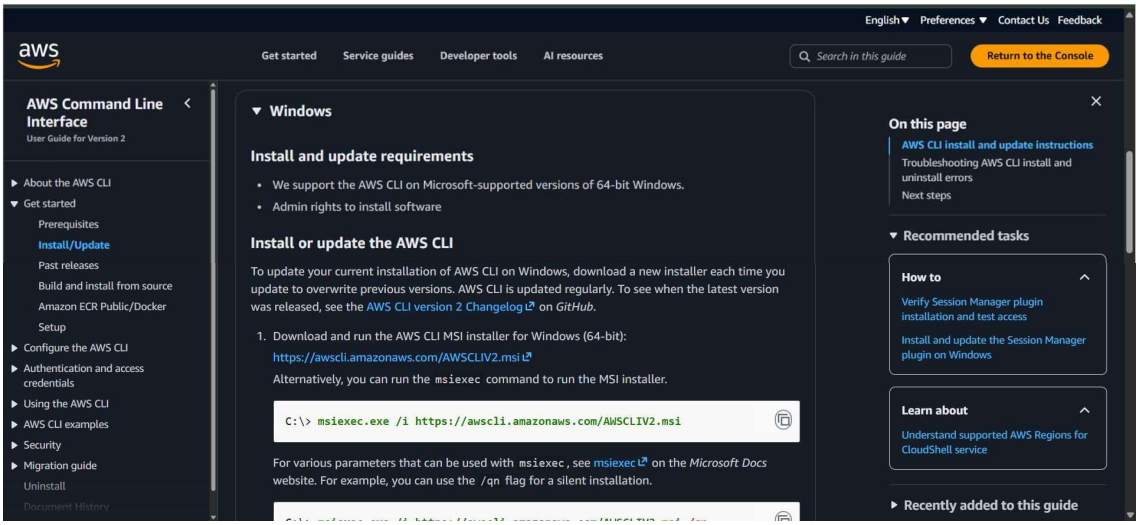
AWS CLI = API caller + credential handler.

2. What "Installing AWS CLI" Means

At a theoretical level, installation involves:

1. Placing the AWS CLI executable on your system
2. Ensuring the OS can locate it via PATH
3. Making required runtime dependencies available (Python is bundled in modern versions)

That's it. No AWS interaction yet.



3. Binary + Runtime Model

Modern AWS CLI v2 ships as:

- A self-contained binary
- With an embedded Python runtime

This matters because:

- You do not need system Python
- You avoid version conflicts
- AWS controls compatibility

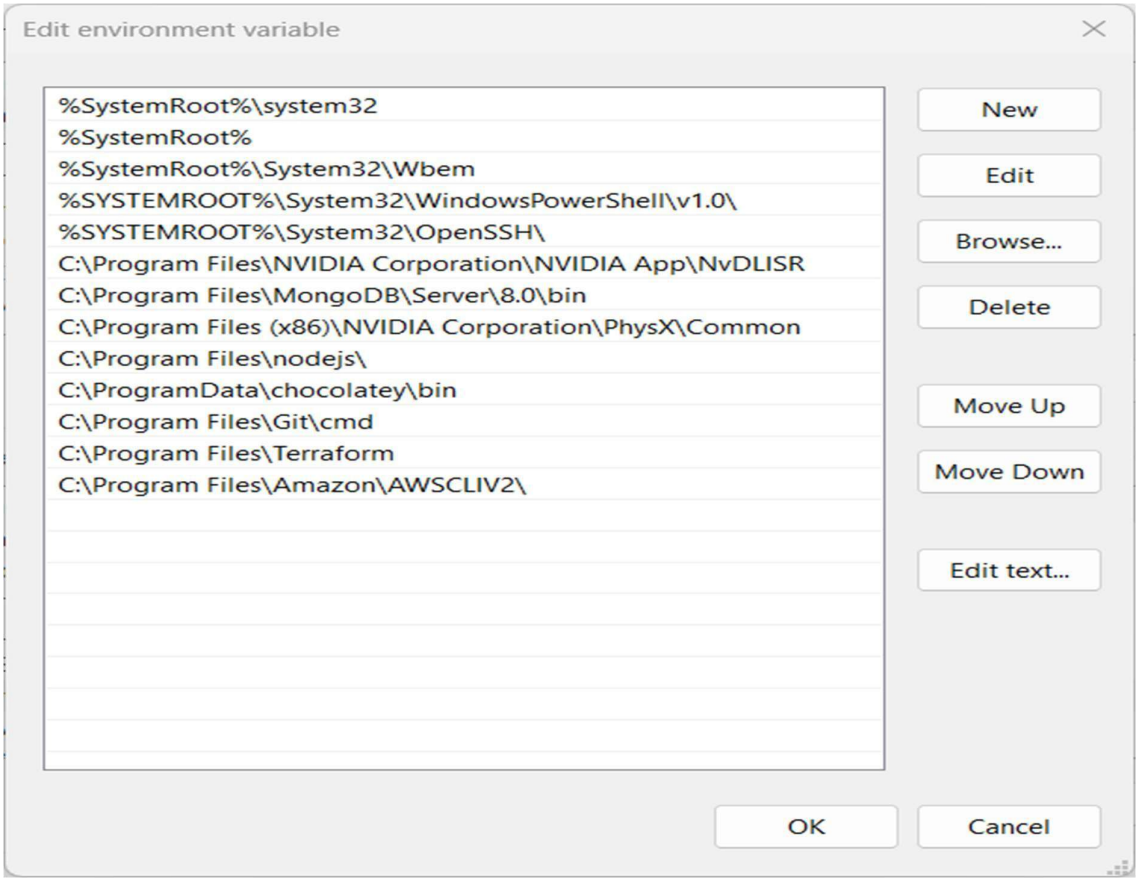
So installation ≠ Python setup. That misconception needs to die.

4. Role of PATH (Again — This Is Fundamental)

Same rule as Terraform:

- OS searches PATH
- Finds aws
- Executes it

If PATH is wrong, AWS CLI “isn’t installed” from the OS’s perspective, even if files exist.



5. Why Package Managers Are Used

Package managers:

- Download the correct AWS CLI binary
- Place it in a standard executable directory
- Update PATH
- Handle upgrades cleanly

They reduce human error, not complexity.

6. Verification — What It Proves

Running a version check conceptually proves:

- The OS can execute the AWS CLI
- The binary is intact
- Dependencies are resolved

It does not prove:

- AWS access
- Correct credentials
- Network permissions

```
C:\Windows\System32>aws --version
aws-cli/2.32.31 Python/3.13.11 Windows/11 exe/AMD64

C:\Windows\System32>aws configure
AWS Access Key ID [None]:
AWS Secret Access Key [None]:
Default region name [None]:
Default output format [None]:
```